

Introduction

Your manager, Tony, wants to make sure that you can detect the widely used LLMNR, NBT-NS and MDNS poisoning tool, Responder. Tony was also informed, after a recent penetration test, that a PowerShell-based Responder variant, called Inveigh, is being used in the wild.

A hunting exercise has been scheduled, where you are tasked with detecting Responder's or Inveigh's presence on the network.

LLMNR, NBT-NS and MDNS poisoning can be seen as a Man-in-The-Middle attack that is similar to ARP poisoning, in terms of the actual outcome. In both LLMNR, NBT-NS & MDNS poisoning and ARP poisoning, the victim will eventually interact with an attacker controlled system. In the case of LLMNR, NBT-NS and MDNS poisoning, the attack may result in the victim's NetNTLM password hash being captured and reused by the attacker.

Following is how LLMNR, NBT-NS and MDNS poisoning is performed, at a high level.

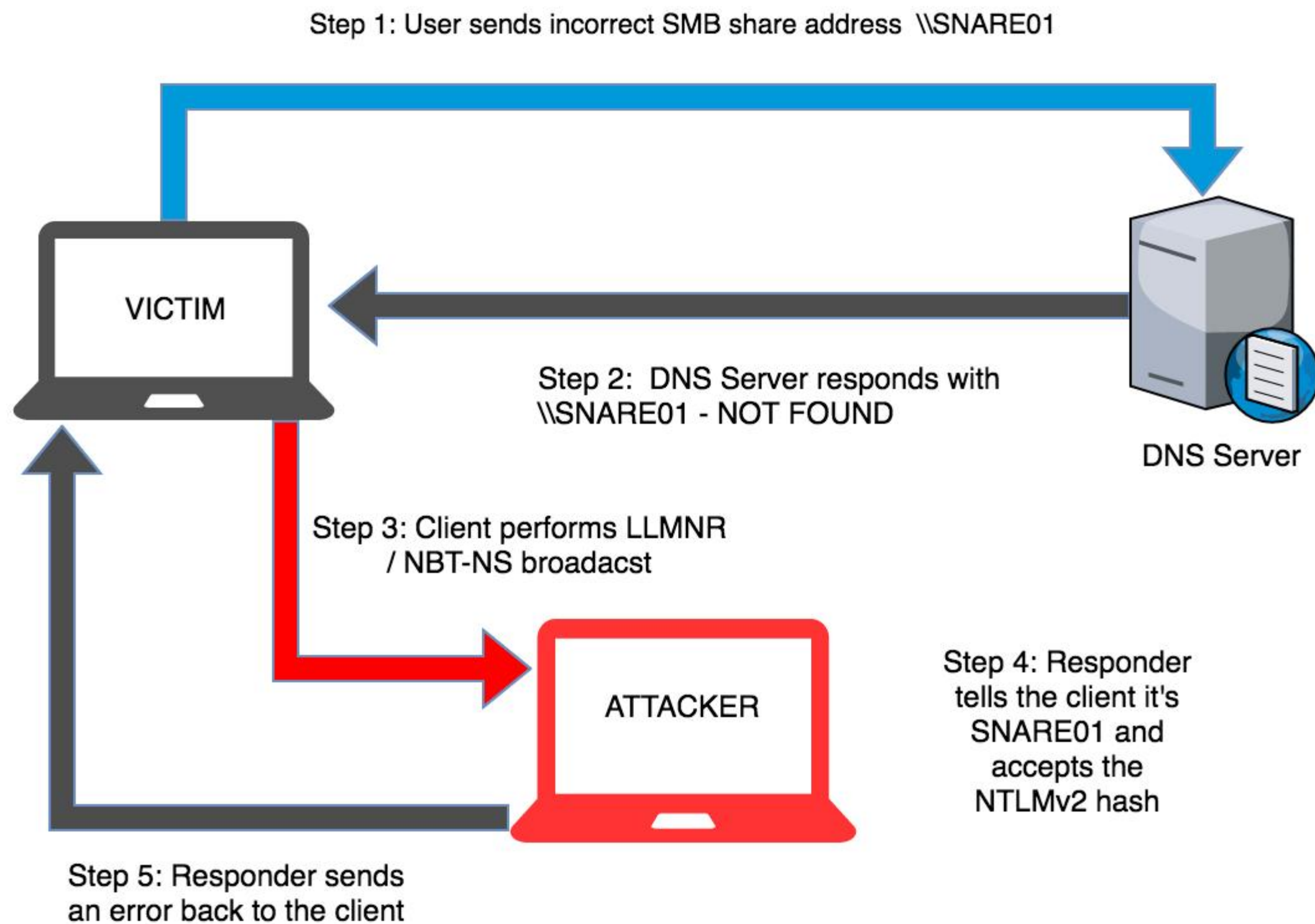


Image by:

<https://www.aptive.co.uk>

Goals

You will need to extend the techniques mentioned in the course to detect Responder or Inveigh operating on a network.

What you will learn

The objective of this lab is to use different detection techniques to detect Responder's or Inveigh's presence on a network. Specifically, you will learn to use CredDefense, Honey Credentials (deliberately placed credentials on the network, acting as bait), Sysmon and PowerShell's capabilities, separately, to detect LLMNR, NBT-NS and MDNS poisoning attempts on a network.

Recommended tools

CredDefense (ResponderGuard module)

Powershell (v3+)

Sysmon

Network Configuration

- SERVER01 (the server from where you will start your hunt):
- IP: 10.100.11.150

Domain Credentials: `INE\ineadmin | JsZkVjx=xQVEEkY8`

Tasks

Task 1: Detect malicious responses and rogue authentication servers inside the network

Responder and Inveigh utilize rogue authentication servers to capture user credentials, after LLMNR, NBT-NS and MDNS poisoning is performed.

By deliberately requesting for a non-existing network resource and using honey credentials, you can detect the presence of Responder or Inveigh inside a network.

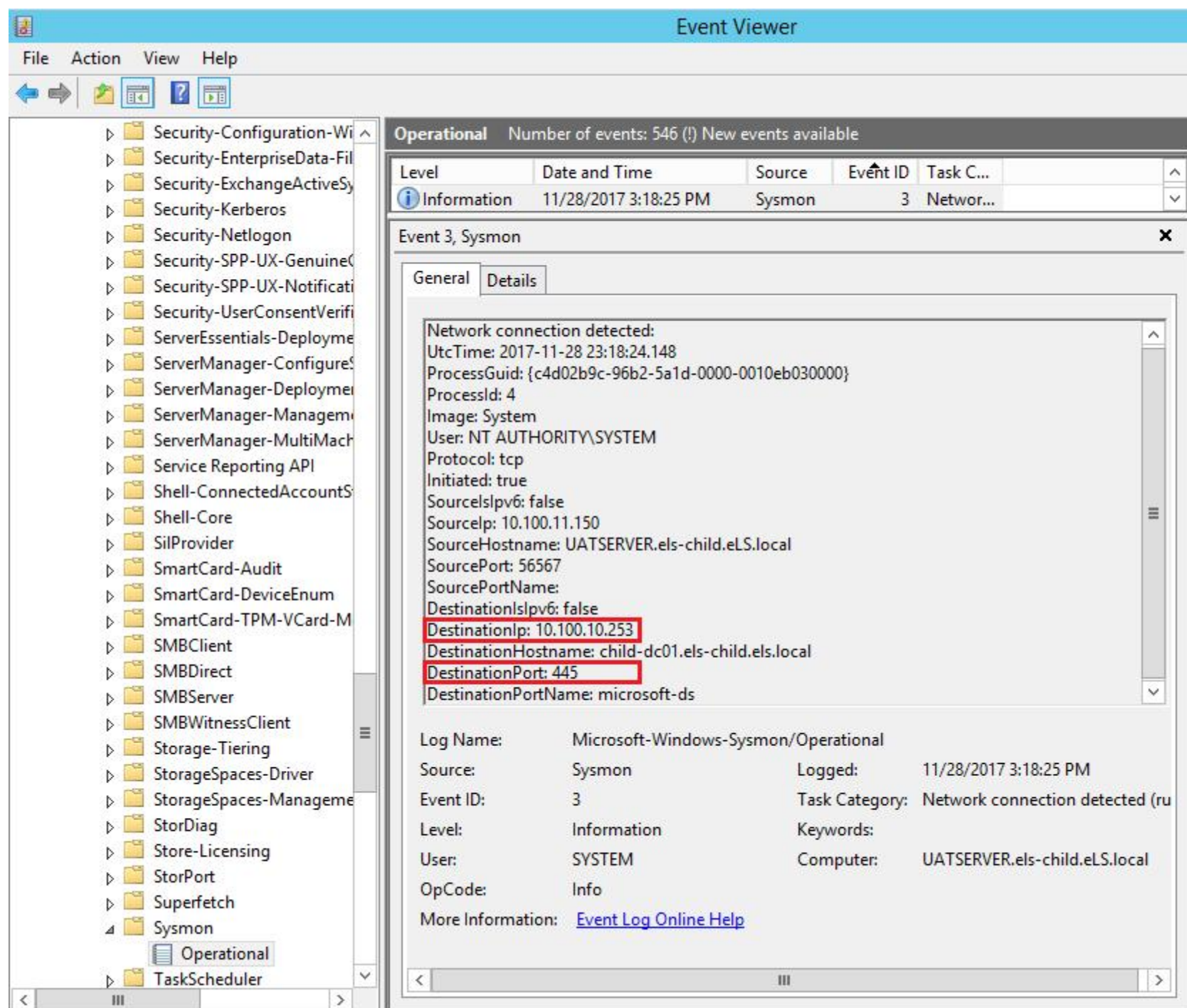
The detection can be performed by looking for any responses regarding the non-existing network resource and also by monitoring the usage of honey credentials.

- Detecting ill-intended responses can be performed by checking if a UDP-based response was returned, for the non-existing network resource you deliberately requested. PowerShell and especially the CredDefense suite can assist you in that.
- Detecting the usage of honey credentials can be performed by analyzing a machine's Security event logs. Specifically, the Event ID 4648 - A logon was attempted using explicit credentials can help you in detecting if any honey credentials were used. PowerShell's Get-EventLog ([https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-powershell-1.0/ee176846\(v=technet.10\)?redirectedfrom=MSDN](https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-powershell-1.0/ee176846(v=technet.10)?redirectedfrom=MSDN)) function will certainly prove handy.

Task 2: Detect suspicious interactions with untrusted IP's

Sysmon is able to collect all interactions of a network asset with other assets in the network. If you query those Sysmon logs for any SMB (or NetBIOS)-related communications with an untrusted IP, you will be able to identify if Responder or Inveigh is operating inside the network.

Specifically, Sysmon's Event ID 3 contains all the information you need to detect any SMB (or NetBIOS)-related communications with an untrusted IP. If the Destination IP is not a trusted File Server or a Domain Controller, chances are that Responder or Inveigh is operating inside the network.



Task 3: Use PowerShell's capabilities to detect Inveigh

Latest PowerShell versions introduced some great blue team-oriented capabilities. One of those capabilities is PowerShell Script Block logging. Script block logging records blocks of code during execution. As a result a script's entire functionality can be recorded and then analyzed. Due to the way script block logging operates, de-obfuscated code can also be recorded, as it is being executed. All logs are recorded into the 'Applications and Services Logs' - Microsoft - Windows - PowerShell - Operational log, under the Event ID 4104.

Inveigh, the PowerShell counterpart of Responder, is considered an extremely effective attacking tool, since it can be executed externally. By enabling PowerShell's script block logging you could easily spot Inveigh running on an endpoint's memory.

Solutions

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

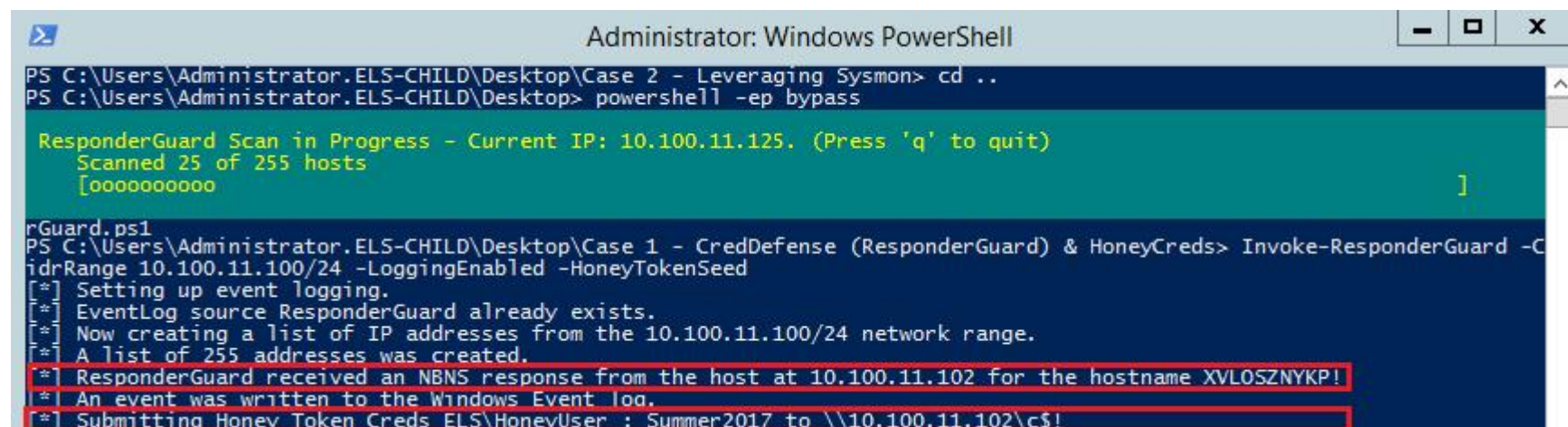
Task 1. Detect malicious responses and rogue authentication servers inside the network

Once you are logged in to the lab, open a PowerShell terminal as Administrator, navigate to the **Case 1 - CredDefense (ResponderGuard) & HoneyCreds** folder, which is located in the Desktop (c:\users\ineadmin\desktop) and execute the following. [ResponderGuard is part the CredDefense suite]

```
powershell -ep bypass
Import-Module .\ResponderGuard.ps1
Invoke-ResponderGuard -CidrRange 10.100.11.0/24 -LoggingEnabled -HoneyTokenSeed
```

If you inspect ResponderGuard's code, you will see that it deliberately requests for a non-existing network resource and listens for any ill-intended responses.

You should see something similar to the below.



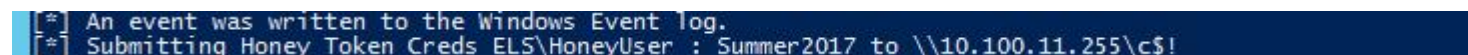
```
Administrator: Windows PowerShell
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 2 - Leveraging Sysmon> cd ..
PS C:\Users\Administrator.ELS-CHILD\Desktop> powershell -ep bypass

ResponderGuard Scan in Progress - Current IP: 10.100.11.125. (Press 'q' to quit)
Scanned 25 of 255 hosts
[oooooooooooo]

rGuard.ps1
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 1 - CredDefense (ResponderGuard) & HoneyCreds> Invoke-ResponderGuard -C
idrRange 10.100.11.100/24 -LoggingEnabled -HoneyTokenSeed
[*] Setting up event logging.
[*] EventLog source ResponderGuard already exists.
[*] Now creating a list of IP addresses from the 10.100.11.100/24 network range.
[*] A list of 255 addresses was created.
[*] ResponderGuard received an NBNS response from the host at 10.100.11.102 for the hostname XVLOSZNYKP!
[*] An event was written to the Windows Event log.
[*] Submitting Honey Token Creds ELS\HoneyUser : Summer2017 to \\10.100.11.102\c$!
```

The above means that Responder or Inveigh is present in the network, since we got a response for a non-existing network resource.

Be prepared for false positives like the one below.

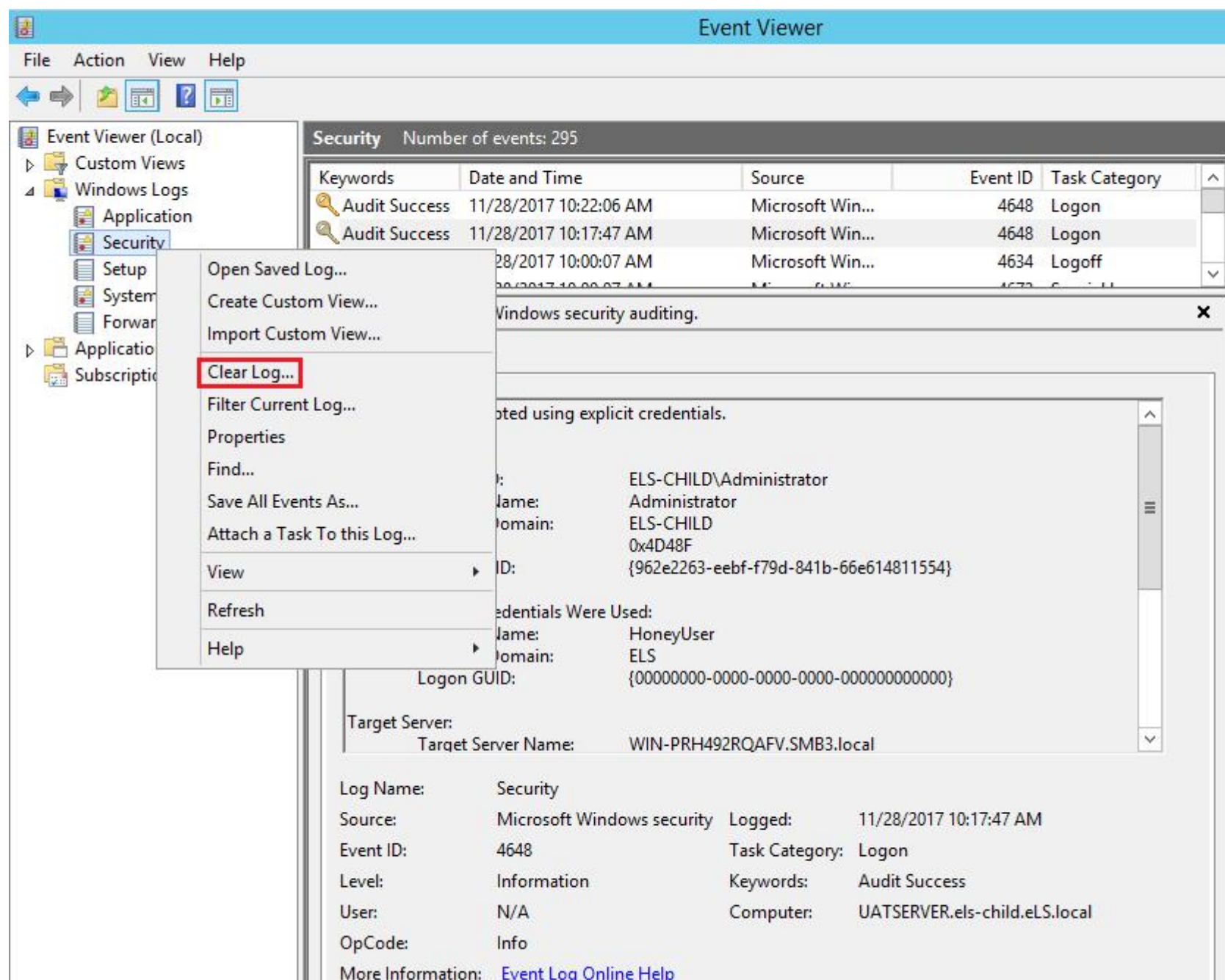


```
[*] An event was written to the Windows Event log.
[*] Submitting Honey Token Creds ELS\HoneyUser : Summer2017 to \\10.100.11.255\c$!
```

ResponderGuard also submits honey credentials to any rogue authentication server of Responder or Inveigh that is detected. So, we can also detect Responder or Inveigh operating inside the network by monitoring the usage of those credentials. As already mentioned the security Event ID that is of interest when looking for such threats, is Event ID **4648 - A logon was attempted using explicit credentials**.

We can monitor and query the logs associated with the Event ID 4648 as follows.

First go the machine's Event Viewer and clear all Security logs.



Then open a PowerShell terminal as Administrator, navigate to the **Case 1 - CredDefense (ResponderGuard) & HoneyCreds** folder, which is located in the Desktop and execute the following. (Or use your existing session if it is still open)

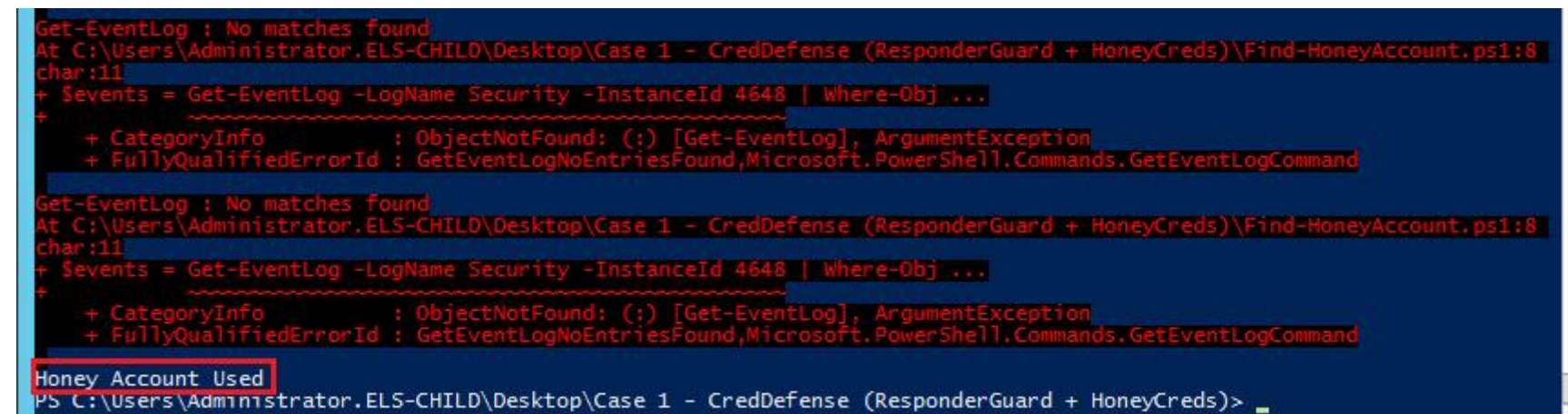
```
powershell -ep bypass
Import-Module .\ResponderGuard.ps1
Invoke-ResponderGuard -CidrRange 10.100.11.0/24 -LoggingEnabled -HoneyTokenSeed
```

and concurrently execute in another PowerShell terminal (run again as Administrator) the below

```
powershell -ep bypass
Import-Module .\Find-HoneyAccount.ps1
```

Find-HoneyAccount HoneyUser

You should see something similar to the following, once the honey credentials are used.



```
Get-EventLog : No matches found
At C:\Users\Administrator.ELS-CHILD\Desktop\Case 1 - CredDefense (ResponderGuard + HoneyCreds)\Find-HoneyAccount.ps1:8
char:11
+ $events = Get-EventLog -LogName Security -InstanceId 4648 | Where-Obj ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (:) [Get-EventLog], ArgumentException
+ FullyQualifiedErrorId : GetEventLogNoEntriesFound,Microsoft.PowerShell.Commands.GetEventLogCommand

Get-EventLog : No matches found
At C:\Users\Administrator.ELS-CHILD\Desktop\Case 1 - CredDefense (ResponderGuard + HoneyCreds)\Find-HoneyAccount.ps1:8
char:11
+ $events = Get-EventLog -LogName Security -InstanceId 4648 | Where-Obj ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (:) [Get-EventLog], ArgumentException
+ FullyQualifiedErrorId : GetEventLogNoEntriesFound,Microsoft.PowerShell.Commands.GetEventLogCommand

Honey Account Used
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 1 - CredDefense (ResponderGuard + HoneyCreds)>
```

Find-HoneyAccount.ps1 content:

```
$Print = "Honey Account Used";
function Find-HoneysAccount {
    Param(
        [String]$AccountName
    )
    while($True) {
        $events = Get-EventLog -LogName Security -InstanceId 4648 | Where-Object { $_.Message -Like "*Account Name:*$($AccountName)*" }
        $events | ForEach-Object {
            $_.Message -match "Account Name:(?<content>.*)$($AccountName)" | Out-Null
            $Print;exit
        }
    }
}
```

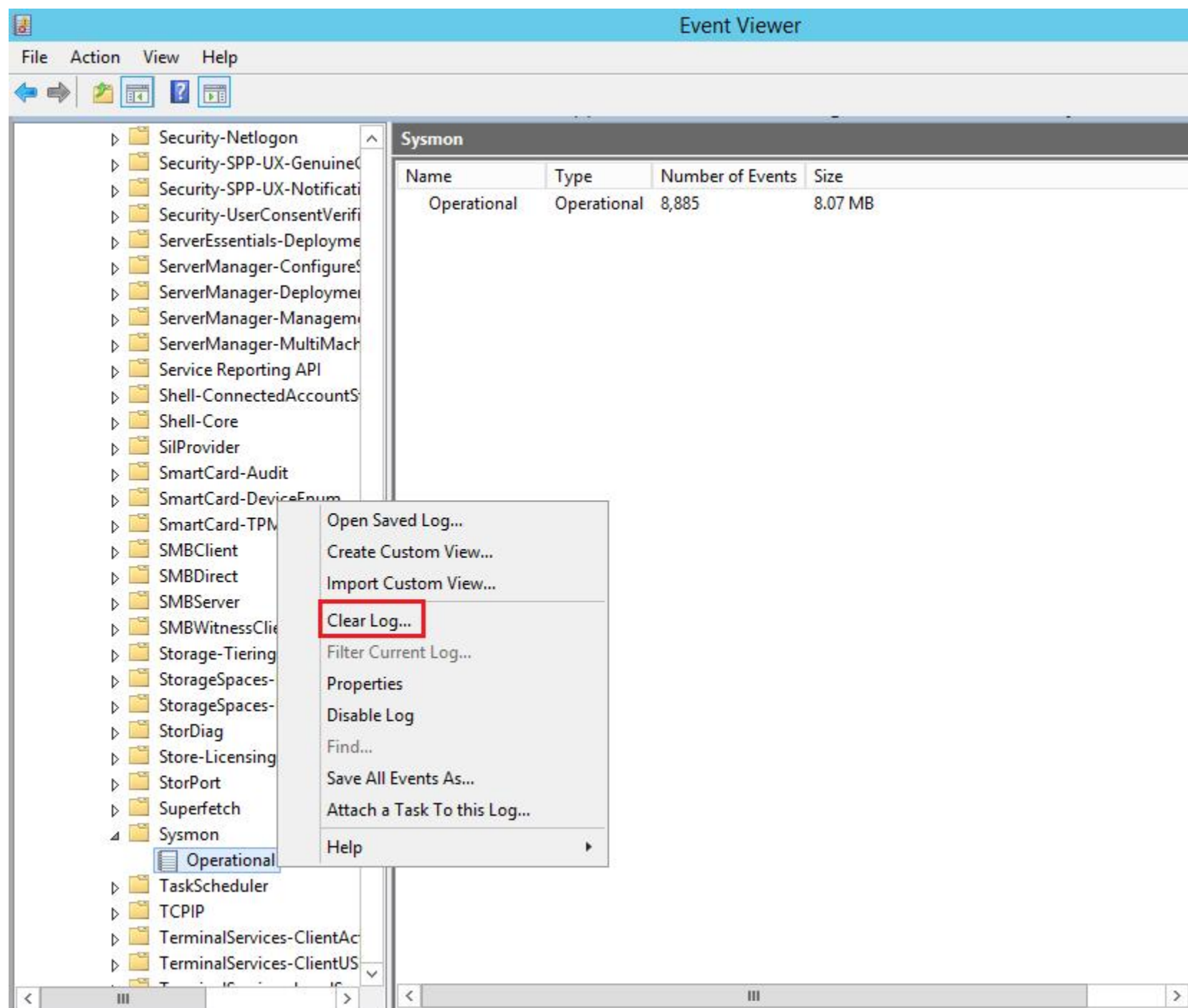
This is how you can query a machine's Security logs using PowerShell

Task 2. Detect suspicious interactions with untrusted IP's

Let's now leverage Sysmon to try and identify any SMB (or NetBIOS)-related communications with an untrusted IP. If the destination IP is an IP other than the one of a trusted file server or a domain controller, then Responder or Inveigh may be operating inside the network.

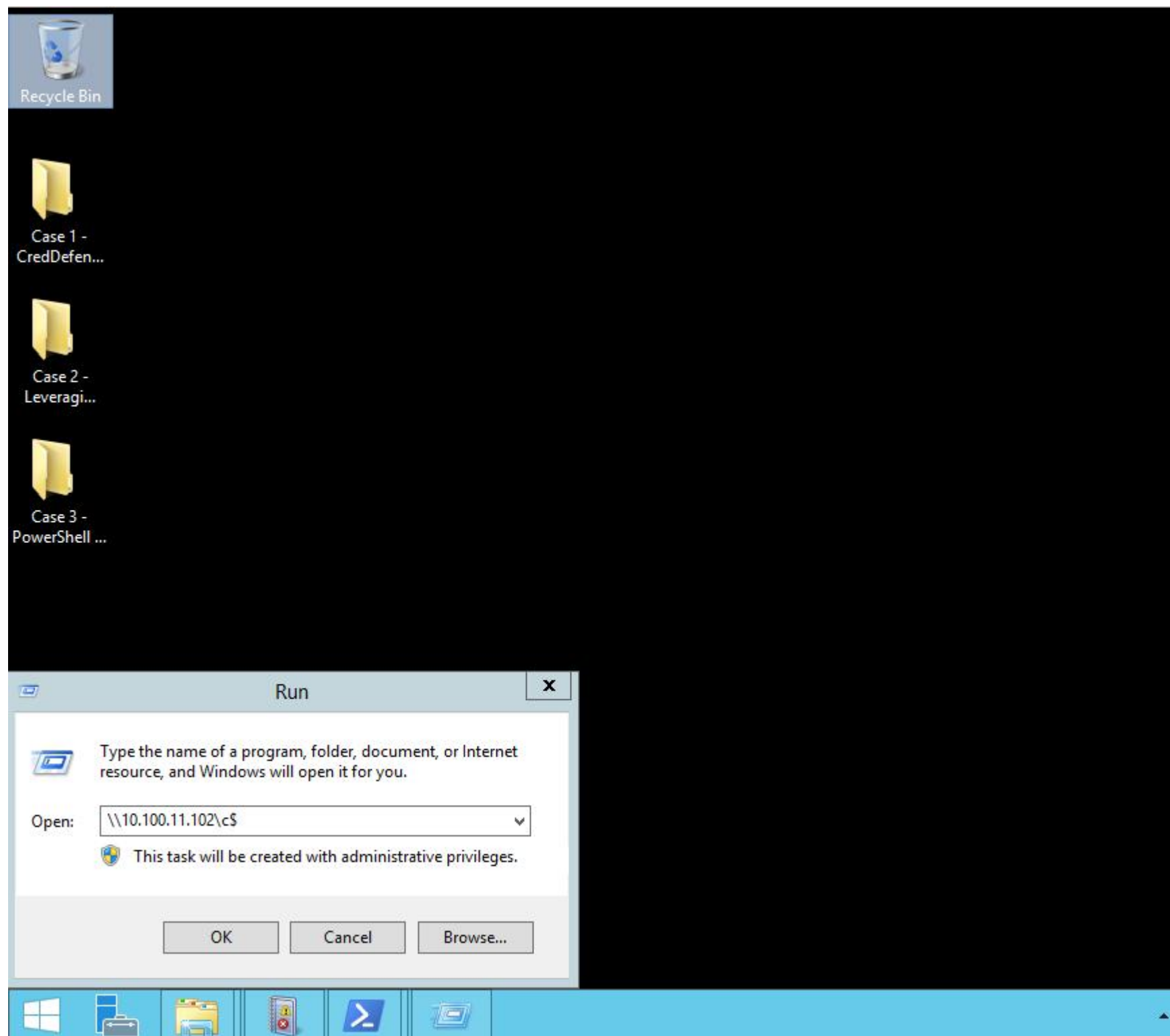
As mentioned, the Sysmon Event ID that is of interest when looking for any SMB (or NetBIOS)-related communications with an untrusted IP, is Sysmon Event ID 3.

First go to the machine's Event Viewer and clear the '**Applications and Services Logs' - Microsoft - Windows - Sysmon - Operational log**



Then, simulate an SMB connection with Responder's rogue authentication server. (This is actually similar to what will happen in the background if Responder is operating inside the network)

Right click on Windows (bottom-right corner) - Run - \\10.100.11.102\\c\$



Finally, open a PowerShell terminal as an Administrator, navigate into the **Case 2 - Leveraging Sysmon** folder, which is located in the Desktop and execute the following.

```
powershell -ep bypass
Import-Module .\Get-WinEventData.ps1
.\Find-UntrustedSMBConnections.ps1
```

You should see something similar to the below.

```
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 2 - Leveraging Sysmon> import-module .\Get-WinEventData.ps1
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 2 - Leveraging Sysmon> .\Find-UntrustedSMBConnections.ps1
SMB Response Sent to Untrusted : 10.100.11.102
SMB Response Sent to Untrusted : 10.100.11.102
SMB Response Sent to Untrusted : 10.100.11.102
SMB Response Sent to Untrusted : 10.100.11.102
SMB Response Sent to Untrusted : 10.100.11.102
SMB Response Sent to Untrusted : 10.100.11.102
SMB Response Sent to Untrusted : 10.100.11.102
```

Find-UntrustedSMBConnections.ps1 content:

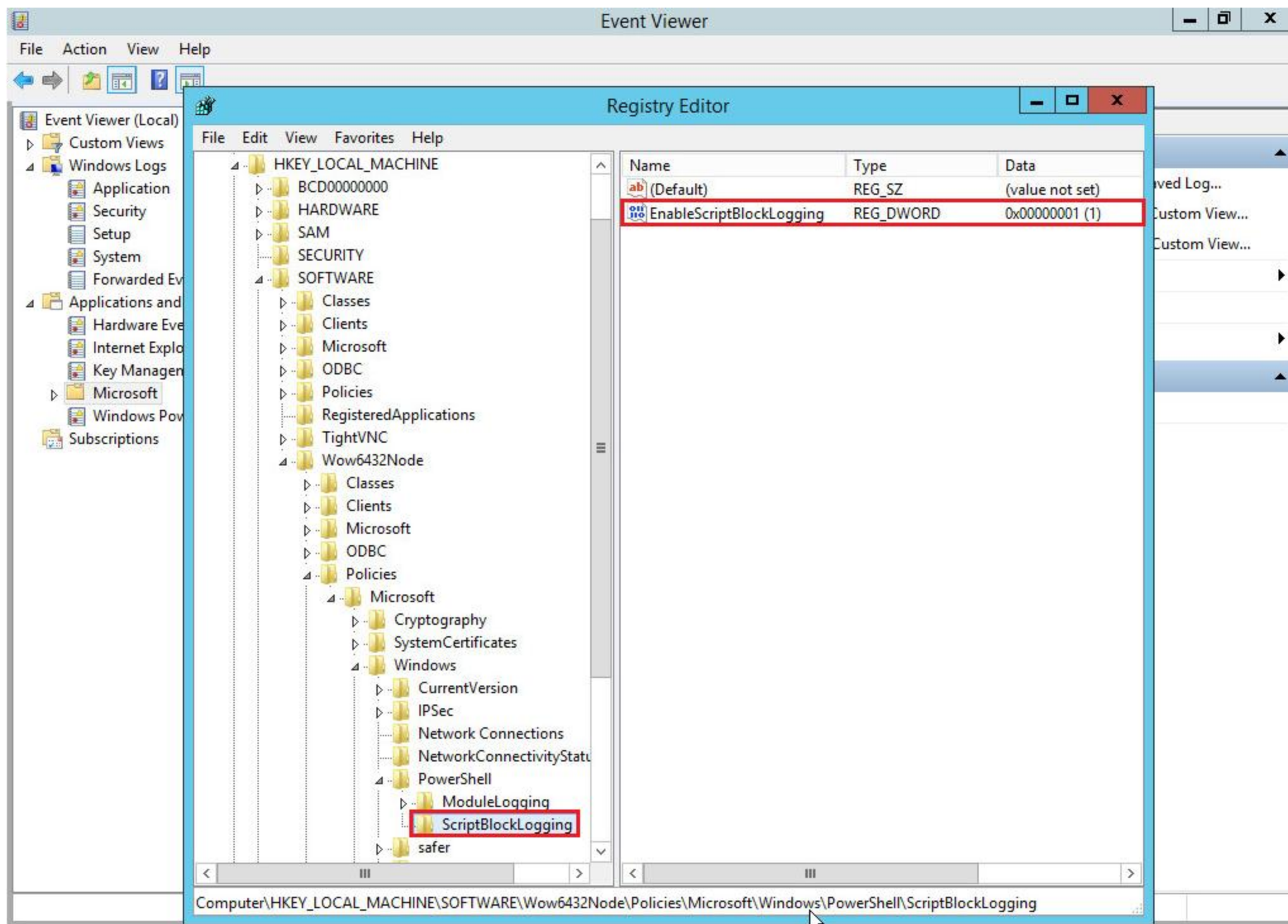
```
#credits to haveyousecured.blogspot.gr
Import-Module Get-WinEventData.ps1
Set-Location \\nonexisting\sharenotthere -ErrorAction SilentlyContinue
$EventsID3 = Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational";id=3} | Get-WinEventData | select EventDataDestinationPort,
EventDataDestinationIp
foreach($Event3 in $EventsID3){
    if(($Event3.EventDataDestinationPort -eq 445) -and ($Event3.EventDataDestinationIp -notcontains "10.100.10.253")){
        Write-Host "SMB Response Sent to Untrusted": $Event3.EventDataDestinationIp
    }
}
```

This is how you can query a machine's Sysmon logs using PowerShell

You could also use the script above for port 139 (NetBIOS).

Task 3. Use PowerShell's capabilities to detect Inveigh

Let's now leverage PowerShell's Script Block Logging capability. You can enable script block logging on a Windows Server 2012 if Windows Management Framework (WMF) 5.0 and .Net 4.5 are already installed and the following registry key and DWORD are configured, as follows.



First, open a new PowerShell terminal as an Administrator, navigate into the **Case 3 - PowerShell for Blue Teamers** folder, which is located in the Desktop and execute the following.

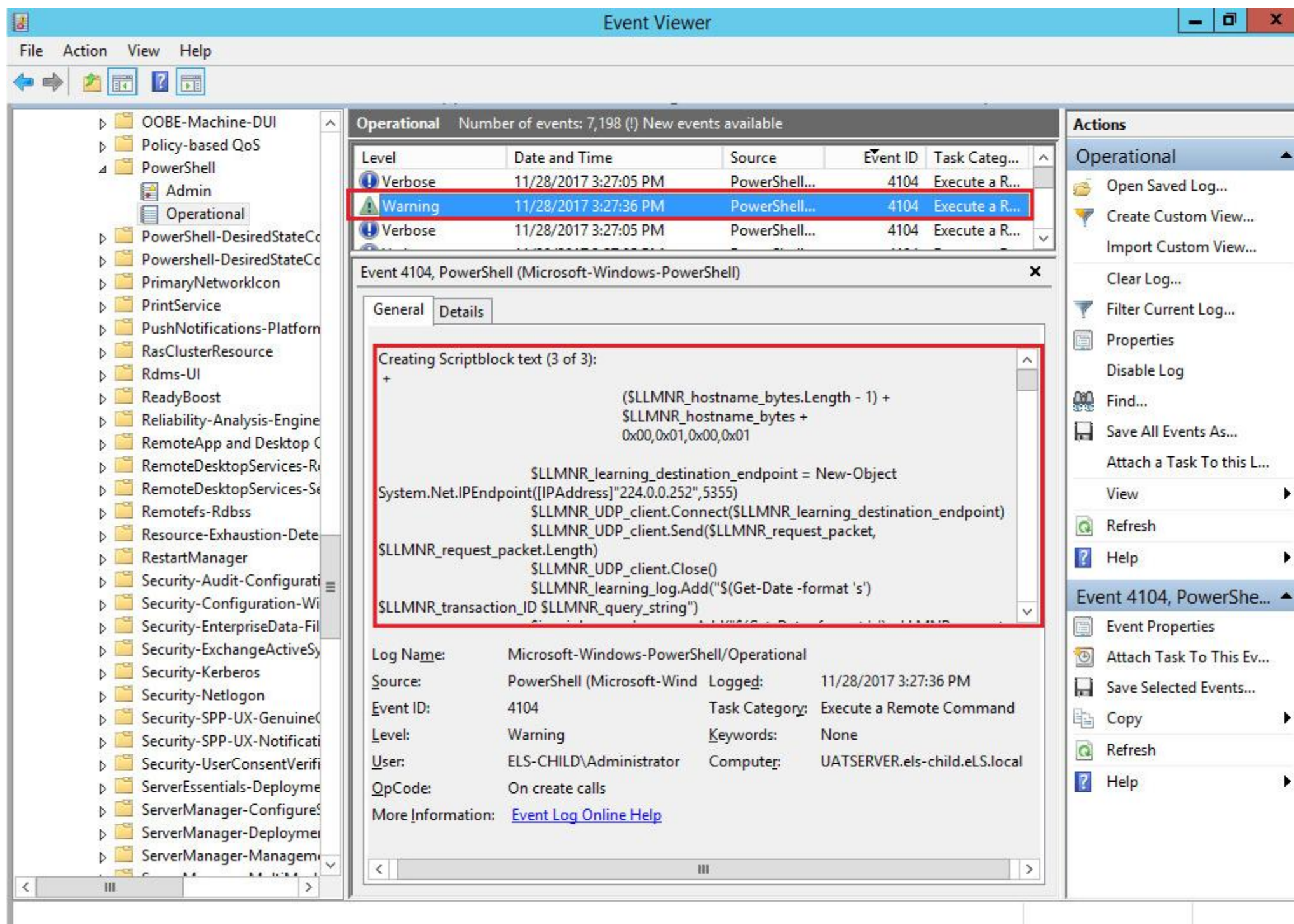
```
powershell -ep bypass
Import-Module .\Inveigh.ps1
Invoke-Inveigh -IP 10.100.11.150
```

You should see something similar to the below.

```
PS C:\Users\Administrator.ELS-CHILD> cd '..\Desktop\Case 3 - PowerShell v3+ Defense'
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 3 - PowerShell v3+ Defense> import-module .\Inveigh.ps1
PS C:\Users\Administrator.ELS-CHILD\Desktop\Case 3 - PowerShell v3+ Defense> Invoke-Inveigh -IP 10.100.11.150
Inveigh 1.3.1 started at 2017-11-28T14:18:39
Elevated Privilege Mode = Enabled
WARNING: Windows Firewall = Enabled
Primary IP Address = 10.100.11.150
LLMNR/mDNS/NBNS Spoofer IP Address = 10.100.11.150
LLMNR Spoofer = Enabled
LLMNR TTL = 30 Seconds
mDNS Spoofer = Disabled
NBNS Spoofer = Disabled
SMB Capture = Enabled
HTTP Capture = Enabled
HTTPS Capture = Disabled
HTTP/HTTPS Authentication = NTLM
WPAD Authentication = NTLM
WPAD NTLM Authentication Ignore List = Firefox
HTTP Reset Delay List = Firefox
HTTP Reset Delay Timeout = 30 Seconds
WPAD Default Response = Enabled
Machine Account Capture = Disabled
Real Time Console Output = Disabled
Real Time File Output = Disabled
WARNING: Run Stop-Inveigh to stop Inveigh
```

Now, open the machine's Event Viewer and navigate to the '**Applications and Services Logs**' - **Microsoft** - **Windows** - **PowerShell** - **Operational log**.

You should see something similar to the below.



The recorded block above is actually part of Inveigh's LLMNR poisoning procedure.

To conclude, in this lab four (4) distinct ways were documented that could assist you in detecting Responder or Inveigh on a network, under your supervision.

- CredDefense (listening for UDP-based responses when requesting a non-existing network resource)

- Honey Credentials (Security Event ID **4648**)
- Sysmon (Sysmon Event ID **3**)
- PowerShell's Script Block Logging (PowerShell Operational log - Event ID **4104**)